

IN3062 Introduction to Artificial Intelligence

By Group 22 - Hamna Ahmad, Edgar Htut, Ramshri Mohapatra, Tharshika Sivananthan



Github link:

https://github.com/Ramshri-Mohapatra/Team22_Intro_To_AI.git

TABLE OF CONTENTS

INTRODUCTION	2
Our Objectives	2
Is the dataset classification or regression?	2
Methods	3
1. Decision Trees	4
2. Neutral Networks	4
3. Support Vector Machines	4
PROCEDURE:	5
1. Cleaning the Dataset	5
2. How we used each method	5
Decision tree	5
Neural Network	6
Process:	6
Hyperparameter Tuning:	6
Testing:	7
Support Vector Machine	7
RESULTS	8
Decision trees	8
Neutral Networks	8
Support Vector Machine	9
Summary of Results	9
REFERENCES	10

INTRODUCTION

In today's world, email communication is essential for both personal and business needs. But effectively handling and arranging the enormous amount of emails that are received every day is a constant struggle. To increase efficiency and guarantee efficient communication, emails must be categorized into groups including spam, personal, and work-related. By examining email content and classifying messages according to recognised patterns, machine learning techniques provide a workable answer to this problem.

The Enron Email Dataset has been chosen as the foundation for analysis in this project. A substantial collection of emails from the Enron Corporation are included in this dataset, which may be accessed at [Enron Email Dataset](<https://www.cs.cmu.edu/~enron/>) and offers actual data for research. The dataset is frequently utilized in machine learning and natural language processing (NLP) research because of its large size, which includes more than 500,000 emails with a range of content, making it ideal for testing and assessment.

The practical significance of this dataset and the difficulties it poses such as managing noise, ambiguity, and massive textual data were the driving forces behind its selection. This project aims to aid in the creation of automated email management systems that improve individual and organizational productivity by grouping emails into relevant categories.

Our Objectives

- Managing Enron database by focusing on 50 emails per inbox due to challenges in cleaning process
- Classifying emails as "spam" and "not spam"
- Cleaning and processing data that is unstructured.
- Developing accurate models to enhance email management.

Is the dataset classification or regression?

The goal of labelling emails as spam or non spam in the Enron email dataset is a **classification** task because it is concerned with predicting discrete, categorical outcomes rather than continuous ones. Classification is the process of assigning data points to preset groups based on their properties, which is exactly what we're doing here: categorising emails into one of two separate classes "spam" or "not spam." This assignment does not require estimating a continuous variable, as is the case in regression issues using numerical values such as prices

or scores. Instead, it addresses a binary decision-making process.

Methods

1. Decision Trees

Decision trees are one of the most effective and intuitive ways to classify emails as spam or not spam. They operate by building a tree where each node is a decision based on a particular feature of the email, like the presence of certain keywords, the domain of the sender, or structural features such as the number of links in the email. At each node of the tree, it splits the dataset into subsets with the criterion that maximizes the distinction between spam and not-spam emails. The splitting is continued until all the subsets are categorized under one of the two final labels, which are either spam or not spam. The key strength of the decision trees is that it can manage diverse features present in the dataset.

The emails can have several attributes such as word frequency, length of the subject line, or metadata like sender information. All these can be assessed well by a decision tree. For example, it may first look for the presence of high-risk keywords like "win" or "free" and then check whether the sender's domain is in a list of suspicious addresses. Each step of the procedure reduces the classification by focusing upon the most predictive features towards the outcome. Another useful benefit of decision trees relates to clarity and interpretability.

Decision trees provide clear visibility of the entire process where decisions are formed, whereas some other types of models work as more of a "black box. This makes it easy to trace why a specific email was classified as spam or not spam, especially in cases where the classification has to be validated or explained. The simple structure also helps in fine-tuning the model because it's easy to see which features and thresholds affect the results of the classification. Another benefit is that they can capture complex relationships between features. For instance, spam emails often have specific combinations of characteristics, such as a keyword-rich body coupled with a suspicious sender address. Decision trees are excellent at identifying these patterns, even when the boundaries between spam and non-spam emails are not obvious. This adaptability is critical for tasks like spam classification, where subtle variations in features can significantly impact the results.

In the case of the Enron email dataset, decision trees are particularly useful since they can process medium-sized datasets in an efficient way to yield a model that is at the same time both accurate and understandable.

2. Neural Networks

Neural network is a classification technique that helps to see if emails are spam or not spam. It shows how the human brain processes information it sees, with layers of nodes that work together to analyse input features such as word selection and sender information as well as structure in emails. Each layer of the network converts the input data into a more abstract representation, allowing for the discovery of subtle relationships that may not be found and for it to perform well in tasks. For example, in our case, separating between categories, such as spam and non-spam which includes analysing diverse and complex data.

Neural networks are also highly effective at adjusting to changes in data patterns, making it desirable for spam recognition. Spam emails have been changing regularly, and spammers use new techniques to get around scanners. Neural networks can also be updated to recognise new patterns using new datasets to ensure its algorithm remains successful. This is an advantage over normal rule-based systems, which can sometimes fail to keep up with fast changing trends.

Overall, neural networks are a good choice for identifying emails in the Enron dataset. It deals with complex data and adapts to new spam techniques. Although this requires careful setup and computer resources, it is accurate and adapts well to latest trends making it a reliable tool for detecting spam.

3. Support Vector Machines

Support Vector Machines (SVMs) are considered an efficient tool for categorising emails as spam or non-spam when the goal is to effectively separate data into groups. SVMs determine the boundary known as the hyperplane, which efficiently splits the dataset into two groups. Spam versus non-spam. This hyperplane is chosen to maximise the margin or space between each group's data points, also known as support vectors. By focussing on and properly analysing these data sets, Support Vector Machines (SVMs) establish a strong ability to foresee previously unknown information, proving to be a dependable solution for effectively detecting spam.

SVMs excel at handling high-dimensional data, which is frequently encountered in email classification jobs. Emails usually show a variety of properties, like word frequencies, information, and structural patterns. SVMs excel at processing this type of data by converting it into a higher-dimensional space via a technique known as the kernel trick.

This improvement allows SVMs to locate a hyperplane even when the data is not easily separable in its original form. For example, emails containing a mix of spam and valid content

can be efficiently sorted in this changed space. Another advantage of SVMs is their ability to prevent overfitting, particularly when utilising a properly adjusted kernel algorithm and regularisation parameter.

This makes them appropriate for datasets such as the Enron emails, which may contain clutter.

PROCEDURE:

1. Cleaning the Dataset

The Enron email dataset was extraordinarily huge and unorganised, including over 500,000 emails. In order to manage the data for this project, we initially decided to limit and only include emails in each individual's inboxes, excluding all other email folders. Looking into the emails we realised that they contained huge amounts of clutter and was going to require intensive cleaning and many individuals contained 1000+ emails so we further decided to limit it to 50 emails per person. Initially we started with manually cleaning the emails and removing headers like message-ID, cc, mime-version, content-type, content-transfer-encoding, x-from, x-to, x-cc, x-bcc, x-Folder, x-origin, x-FileName, signatures and footers. Due to the emails containing a large amount of junk it was taking extremely long to clean it manually hence we worked on creating a script which aided us in cleaning the database more quickly.

2. How we used each method

Decision tree

Process:

Decision Trees classify emails as spam or not spam by creating a hierarchical structure of decisions. The text data is first transformed into numerical features using TfidfVectorizer, which converts the email content into vectors based on the frequency and importance of words. To ensure balanced data, SMOTE (Synthetic Minority Oversampling Technique) is applied, which helps handle class imbalance by generating synthetic examples of the minority class, additionally class weight is set to balanced and gini criterion was selected to further address imbalance data. The DecisionTreeClassifier from scikit-learn is then used to train a model that splits the dataset based on feature importance. Each split aims to maximize information gain, leading to a structure where the leaves represent the final decision (spam or not spam). Testing with different hyperparameters showed minimal accuracy differences, the model performs consistently. Max depth was set to 20 even though larger values gave slightly better accuracy, for a more generalized model and reduced risks of overfitting. Min samples split and leaf were both set to 5 for the same reason, as lower values would lead to overfitting especially on smaller testing datasets.

Testing :

The model's effectiveness is measured using metrics such as accuracy, recall, and F1-score to determine how well it differentiates between spam and non-spam emails. Confusion matrices provide precise information into true positives, false positives, and other essential indicators, making it simple to discover potential problems. The evaluation procedure entails analysing results and comparing performance across multiple datasets using cross-validation to verify consistency. Testing determines whether the model is overly specialised or capable of adapting to new data.

Neural Network

Process:

The process first begins with data loading and preparation. Emails are loaded from a specified directory which segregates its training and testing datasets. Labels are assigned based on predetermined spam keywords with a binary classification where 1 means spam and 0 means non-spam. The data is preprocessed by extracting features using TF-IDF vectorization, which converts the email text to numerical representation by emphasizing unique words and placing less focus on common words. Class balance is also addressed using SMOTE, an oversampling technique that generates synthetic samples for the minority class (spam).

A neural network was established using TensorFlow's Keras API, featuring a three-layer architecture with dropout layers to prevent overfitting and L2 regularization to enhance robustness. The training process spanned upto 200 epochs, with early stopping implemented to monitor validation loss and optimize model performance while avoiding overfitting.

Hyperparameter Tuning:

To refine the model's performance, multiple configurations of hyperparameters were tested. The tuning involved varying learning rates, patience levels for early stopping, and the number of epochs to achieve optimal results. The following changes were explored:

- **Learning Rate:** Tested values of 0.001 and 0.0001 for the Adam optimizer. The smaller learning rate demonstrated better convergence.
- **Patience for Early Stopping:** Adjusted from 2 to 10 epochs, with the final model achieving the best results with patience set to 10.
- **Number of Epochs:** Initially set to 100 and increased to 200 in the final configuration, enabling the model to learn effectively.

- **Batch Size:** Consistently set to 32 for stable gradient updates.

The final model used a learning rate of 0.0001, patience of 10 for early stopping, and was trained for a maximum of 200 epochs with dropout layers and L2 regularization.

Testing:

The model was evaluated on unseen test data, with predictions compared against true labels to compute accuracy. A confusion matrix provided insight into false positives, false negatives, precision, recall, and F1-score metrics, offering a detailed performance assessment. Additionally, K-fold cross-validation validated the model's stability across different data splits, reporting average and variability in accuracy, precision, recall, and F1-score to ensure consistency and generalizability.

Support Vector Machine

Process:

This classification begins with data loading and preparation. The emails are loaded into training and testing datasets, assigning labels to them based on the appearance of any spam-related keywords. The email content is then vectorised using TF-IDF which converts the text into numerical representation by emphasising its unique terms and reducing the importance of common ones. SMOTE is used to address class imbalance, its applied to oversample the minority class, which creates a synthetic sample of spam emails. Additionally class weight is set to balanced to further address imbalance data, and the rbf kernel to capture non linear patterns in the email data. The model is trained on resampled data and tested on a separate dataset to evaluate how it is able to classify unseen emails. The regularization parameter C was set to 10, balancing model complexity and misclassification penalties, lower values underfit, while higher values overfit. Similarly the gamma value is set to 0.1 for better generalization and flexibility, as smaller values would underfit and larger values would overfit.

Testing:

The testing for this model begins with vectorising the test data using the pre-fitted TF-IDF vectoriser, The trained model then predicts labels for the test set and the predictions are compared against true labels to compute accuracy, recall and F1-score. A confusion matrix is also generated to visualise raw and normalised forms to evaluate its classification performance, A K-fold cross validation is also performed to measure the generalisability of different training data.

RESULTS

Decision trees

Decision Tree Accuracy on Test Data: 0.92
(cleaned_emails_nn)

Recall: 0.81

F1-Score: 0.88

K-Fold Results:

Fold #1, Accuracy: 0.94

Fold #2, Accuracy: 0.92

Fold #3, Accuracy: 0.93

Fold #4, Accuracy: 0.94

Fold #5, Accuracy: 0.91

Neural Networks

Neural Network Accuracy on Test Data: 0.87
(cleaned_emails_nn)

K-Fold Cross-Validation Average Metrics:

Average Accuracy: 0.98 ± 0.00

Average Precision: 1.00 ± 0.00

Average Recall: 0.97 ± 0.01

Average F1-Score: 0.98 ± 0.00

Support Vector Machine

SVM Accuracy on Test Data: 0.87 (cleaned_emails_nn)

Recall: 0.77

F1-Score: 0.80

K-Fold Results for SVM:

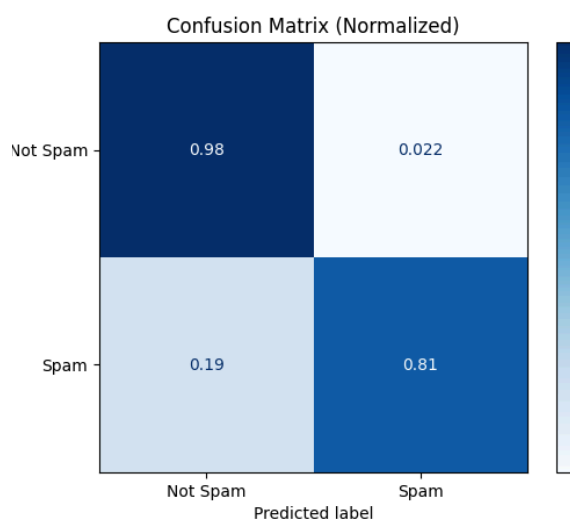


Figure 1: Confusion Matrix for Decision trees

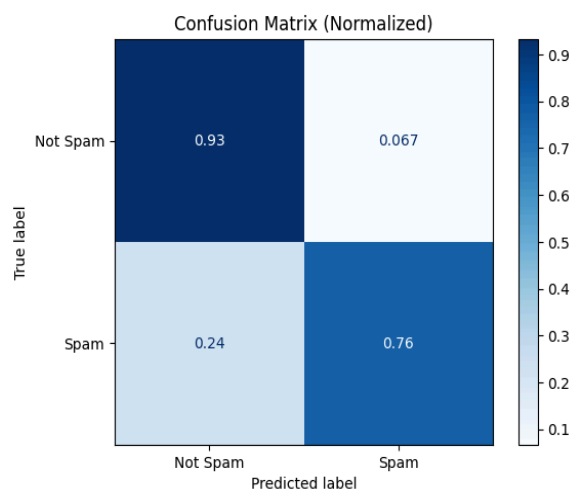


Figure 2: Confusion Matrix for Neural networks

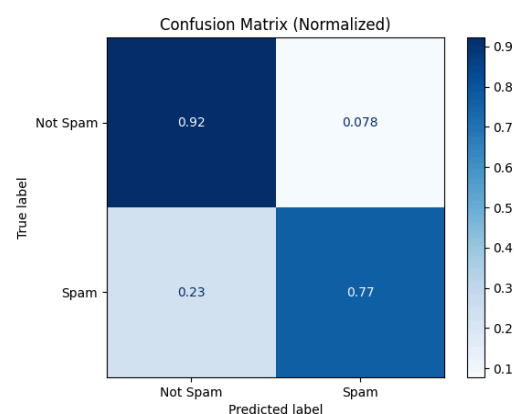


Figure 3: Confusion Matrix for SVMs

Fold #1: Accuracy: 0.89

Fold #2: Accuracy: 0.89

Fold #3: Accuracy: 0.89

Fold #4: Accuracy: 0.90

Fold #5: Accuracy: 0.89

Summary of Results

Each model highlights its unique strengths in spam email classification. Decision trees were the best in terms of accuracy with 92% on the test set and had consistent K-fold results of 92-93%. However, their lower recall shows that it did have challenges in identifying spam. Neural networks had an 87% in test accuracy but as it has a high recall of 97% and F1-score of 98% it was more effective in handling complex data. SVM had 87% accuracy, with lower recall and F1-score giving a reliable but less robust performance. We can conclude that neural networks are the best for balanced classification whilst decision trees are more accurate with raw data.

If we had more time, we would have used more time to clean the data and use more of it in our models. Additionally we would have used NLP and regex for labeling our emails and also data augmentation to artificially expand the size of dataset.

CONCLUSION

To conclude this experiment proved how AI technologies such as neural networks, decision trees, and support vector machines successfully label emails as spam or non spam. Using the Enron email dataset, we studied how these algorithms employ information such as keywords, sender information, and email structure to give more accurate results. Every method has advantages. Decision trees are straightforward to grasp, whereas neural networks are excellent at recognising complex patterns and adjusting to new types of spam.

This work emphasizes on choosing the correct strategy depending on the dataset and problem demands. It also emphasizes the need for constant updates in order to stay up with the changing nature of spam. Overall, this experiment shows how AI can create smarter, more reliable spam filters, which contribute to safer email communication.

REFERENCES

1. Andy (2024). *What are Neural Networks, and How Do They Mimic the Human Brain's Functionality?* [online] Medium. Available at: <https://medium.com/trodyai/what-are-neural-networks-and-how-do-they-mimic-the-human-brains-functionality-69f589936ecc> [Accessed 22 Dec. 2024].
2. Drucker, H., Donghui Wu and Vapnik, V.N. (1999). Support vector machines for spam categorization. *IEEE Transactions on Neural Networks*, 10(5), pp.1048–1054. doi:<https://doi.org/10.1109/72.788645>.
3. Scikit-learn Developers. *TF-IDF Vectorization with TfidfVectorizer*.
4. Chawla, N. V., et al. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*.
5. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
6. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. *Journal of Machine Learning Research*, 12, 2825–2830.
7. Brownlee, J. (2017). *Gentle Introduction to the Adam Optimization Algorithm for Deep Learning*. [online] Machine Learning Mastery. Available at: <https://machinelearningmastery.com/adam-optimization-algorithm-for-deep-learning/>.
8. Gupta, A. (2021). *A Comprehensive Guide on Deep Learning Optimizers*. [online] Analytics Vidhya. Available at: <https://www.analyticsvidhya.com/blog/2021/10/a-comprehensive-guide-on-deep-learning-optimizers/>.
9. Sagar, A. (2019). *5 Techniques to Prevent Overfitting in Neural Networks*. [online] KDnuggets. Available at: <https://www.kdnuggets.com/2019/12/5-techniques-prevent-overfitting-neural-networks.html>.